Migration Report: Repository Layer

Assignment 02: Software Evolution | ExpenseMate, Personal Expense Manager | November 2026

Team: Afshal Liaquat (24K-2558) | Bisma Shahid (24K-3012)

Current state (before)

In Assignment 01, DatabaseManager (database.py) handled the connection, created the tables, ran all the SQL (9 SQL sites), called model validation, saved changes, and turned rows into models. ExpenseService, AnalyticsEngine, and DataPortabilityManager all depended directly on this class.

- Services loaded every transaction and filtered in Python memory, for example once per budget in get_all_budget_statuses().
- The schema (tables transactions and budgets, indexes idx_trans_date and idx_trans_cat) was created at start-up, with no repository layer and no versioned migration procedure.

Target state (after)

Migration path: direct SQL / database-manager access -> repository (data-access) layer. Two classes were added in expense_mate/repositories.py:

- TransactionRepository: add(), get_all(), get_by_month(), get_by_filter().
- BudgetRepository: save(), get(), get_for_month().

All SQL is parameterized and kept inside these classes. DatabaseManager now only provides the connection (as a context manager) and manages the schema. The SQLite database and its schema are unchanged.

Reason for migration

The required architecture (Controller -> Services -> Repository -> Database) cannot exist without a repository layer. Services can now depend on repository objects instead of a concrete database manager, which allows isolated testing and swapping the storage technology. Monthly queries (get_by_month()) replace repeated load-all-and-filter work, and SQL is no longer scattered across mixed-purpose code.

Migration steps

1. Saved the Assignment 01 state as a baseline snapshot tagged assignment-01-baseline (the original commit could not be recovered because the supplied folder had no Git data).
2. Created repositories.py with TransactionRepository and BudgetRepository.
3. Moved the transaction and budget SQL from DatabaseManager into the repositories.
4. Kept thin compatibility wrappers in DatabaseManager (insert_transaction, get_all_transactions, upsert_budget, etc.) that pass work to the repositories, so Assignment 01 callers and tests still work.
5. Left table and column definitions unchanged, so no data transformation was needed.
6. Added test_repositories.py, ran the regression suite, and recorded the work in CHANGELOG.md (commit 5ab2780).

Risks, compatibility, and rollback

- Existing code calling DatabaseManager could break: wrappers were kept, and the 9 Assignment 01 tests run unmodified.
- Stored data could be changed or lost: schema and columns are unchanged, existing database files stay compatible, and tests use temporary databases, never the original file.
- Wrappers add extra methods to database.py: accepted to avoid breaking old callers; the wrappers contain no SQL.
- No schema-version table or rollback script: not needed because the schema did not change, and recorded as a limitation. Rollback is by reverting to the baseline tag.

Verification and acceptance criteria

Repeatable check: python -m unittest test_repositories test_expense_mate -v, run against temporary databases.

- Results: 11 tests, all passing (9 Assignment 01 tests and 2 repository tests, test_transaction_repository_preserves_records and test_budget_repository_updates_existing_monthly_budget).
- The full package of 29 tests passes after later layers were added (see the Testing Report).

Accepted: existing records stay readable, table and index definitions are unchanged, repository and regression tests pass, and SQL is isolated in the repositories.